Reinforcement Learning — Homework 4: DPO

Jacky Hsu

June 2026

HW4 — Problem 1: Principles of DPO and the DPO Trainer

Source references for Part (b) are from the TRL repository, `trl/trainer/dpo_trainer.py` and `trl/trainer/dpo_config.py`, **main branch as of June 2026**. Exact line numbers drift slightly between commits, so I cite the relevant *functions* and quote the operative code.

(a) Exact gradient of the DPO loss and its intuition

Setup. Define the implicit (DPO) reward of a response y under prompt x:

$$\hat{r}_\theta(x, y) \;=\; \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

Let the per-example logit be

$$u(x, y_w, y_l) \;=\; \hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l) \;=\; \beta\left(\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right).$$

Then the loss for a single triple is $\ell = -\log \sigma(u)$.

Derivation. Using $\dfrac{d}{dz} \log \sigma(z) = \sigma(-z) = 1 - \sigma(z)$ and the chain rule,

$$\nabla_\theta \ell \;=\; -\sigma(-u)\, \nabla_\theta u.$$

Since π_{ref} does not depend on θ,

$$\nabla_\theta u \;=\; \beta(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)).$$

Noting that $\sigma(-u) = \sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$, the full gradient is

$$\boxed{\nabla_\theta \mathcal{L}_{\text{DPO}} = -\,\beta\, \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}\Big[\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{weight: higher when model is wrong}} \;\; (\underbrace{\nabla_\theta \log \pi_\theta(y_w \mid x)}_{\uparrow \text{ push up chosen}} - \underbrace{\nabla_\theta \log \pi_\theta(y_l \mid x)}_{\downarrow \text{ push down rejected}})\Big].}$$

Intuition (cf. Section 4 of Rafailov et al., 2023). Gradient descent on this loss does two things per example: it **increases** the log-likelihood of the preferred response y_w and **decreases** that of the rejected response y_l. Crucially, each example is scaled by the weight $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$, which is the probability the *implicit reward model* assigns to the **wrong** ordering. When the current model already ranks the pair correctly (margin large and positive) the weight $\to 0$ and the example contributes almost nothing; when the model ranks the pair **incorrectly** (rejected scored higher than chosen) the weight $\to 1$ and the update is large. So DPO automatically focuses learning on the examples it currently gets wrong, and the β factor controls the overall step magnitude (and how far the policy is allowed to drift from π_{ref}). This adaptive weighting is exactly what prevents the naive "maximize chosen / minimize rejected" objective from degenerating.

(b) Reading the `DPOTrainer` implementation

Question 1 — How a single scalar log-probability per sequence is produced

Two stages cooperate:

1. **Collation** — `DataCollatorForPreference.torch_call()` (`dpo_trainer.py` $\approx$ Line 150). The prompt and each completion are tokenized *separately*, then concatenated so that prompt tokens come first:

   ```
   prompt_chosen_ids   = [ex["prompt_ids"] + ex["chosen_ids"]   for ex in examples]
   prompt_rejected_ids = [ex["prompt_ids"] + ex["rejected_ids"] for ex in examples]
   ```

   Because tokenization is done per-segment, the trainer knows the exact boundary between prompt and completion and builds a **completion mask** that is `1` only on response tokens (and `0` on prompt and padding tokens).

2. **Forward + reduction** — `compute_loss()` / `_compute_loss()` (`dpo_trainer.py` $\approx$ Line 1190). Chosen and rejected are stacked into one batch and run through the model. Per-token log-probabilities of the *realized* next token are gathered, the prompt/padding positions are zeroed using the completion mask, and the remainder is **summed over the sequence dimension**:

   ```
   per_token_logps = selective_log_softmax(shift_logits, shift_labels)
   per_token_logps[shift_completion_mask == 0] = 0.0   # drop prompt + padding
   logps = per_token_logps.sum(dim=1)                  # one scalar per sequence
   ```

   `selective_log_softmax` does a numerically stable `log_softmax` and gathers the entry of the true label, avoiding materializing the full softmax. The result `logps` is a vector with one entry per sequence — i.e. $\log \pi_\theta(y \mid x)$ over **response tokens only**, which is exactly what the DPO loss requires.

Question 2 — What survives truncation when `prompt+response` exceeds `max_length`

Truncation is applied to the *concatenated* `prompt+completion` sequence inside `torch_call()` (`dpo_trainer.py` $\approx$ Lines 152–165), controlled by `truncation_mode`:

```
if self.truncation_mode == "keep_start":
    sl = slice(None, self.max_length)     # keep the FIRST max_length tokens
elif self.truncation_mode == "keep_end":
```

```
    sl = slice(-self.max_length, None)    # keep the LAST  max_length tokens
prompt_chosen_ids = [ids[sl] for ids in prompt_chosen_ids]
```

The **default is `truncation_mode="keep_start"`** (confirmed in `dpo_config.py`: `truncation_mode: str = field(default="keep_start", ...)`). With `keep_start`, the slice `[:max_length]` keeps the **beginning** of the sequence and discards the tail.

Since the prompt sits at the beginning of the concatenated sequence, **the prefix survives, so response tokens at the end are cut first.** If the prompt itself is shorter than `max_length` (the usual case), the whole prompt is preserved and only the response tail is truncated. If the prompt alone already exceeds `max_length`, then even `keep_start` can only keep the first `max_length` prompt tokens and the response may disappear entirely. Choosing `keep_end` inverts the preference: it keeps the end of the prompt+response pair, so it may drop the front of the prompt while preserving later response tokens.

Question 3 — Numerical stability of $\log\sigma(\cdot)$

A literal implementation `torch.log(torch.sigmoid(x))` is unstable: for very negative `x`, `sigmoid(x)` underflows to `0` and `log(0) = -inf`. In `compute_loss()` (`dpo_trainer.py` $\approx$ Line 1280) `DPOTrainer` instead uses PyTorch's fused, numerically stable primitive `F.logsigmoid`:

```
per_sequence_loss = -F.logsigmoid(self.beta * delta_score)
```

`F.logsigmoid(x)` is computed via the softplus form $\log\sigma(x) = -\log(1 + e^{-x}) = -\text{softplus}(-x)$, which internally branches on the sign of `x` so neither e^{x} nor e^{-x} ever overflows. This keeps the loss (and its gradient) finite and accurate across the full range of reward margins.

Question 4 — What IPO actually does in implementation

IPO (Identity/Implicit Preference Optimization, Azar et al.) replaces the log-sigmoid classification loss with a **squared-error regression** toward a fixed target. In the `loss_type == "ipo"` branch (`dpo_trainer.py` $\approx$ Lines 1288–1299):

```
chosen_avg_score   = chosen_scores   / chosen_mask.sum(dim=1).clamp(min=1.0)
rejected_avg_score = rejected_scores / rejected_mask.sum(dim=1).clamp(min=1.0)
ipo_delta = chosen_avg_score - rejected_avg_score
per_sequence_loss = (ipo_delta - 1 / (2 * self.beta)) ** 2
```

Two differences from vanilla DPO: (i) the chosen/rejected log-ratios are **length-normalized** (divided by the number of completion tokens), removing the length bias of summed log-probs; and (ii) the objective is a **squared loss** that drives the margin toward the constant target $\frac{1}{2\beta}$ rather than pushing it to $+\infty$. Because DPO's $-\log\sigma$ keeps rewarding ever-larger margins, it can over-fit / push π_θ arbitrarily far from π_{ref} when preferences are near-deterministic; IPO's bounded regression target prevents this, giving a more controlled KL deviation.

Question 5 — How the reference policy is handled in practice

The reference policy π_{ref} is the frozen anchor in the reward $\hat{r}_\theta$. In `__init__()` (`dpo_trainer.py` $\approx$ Lines 570–599 for the model setup, $\approx$ Lines 762–777 for the dropout/precompute handling) the trainer picks one of three strategies:

1. **Explicit `ref_model`** — if the user passes one, it is used directly (frozen).

2. **Auto-created frozen copy (the common case, and what this HW uses)** — if `ref_model is None` and the model is *not* a PEFT model and `precompute_ref_log_probs` is `False`, the trainer instantiates a separate, frozen copy of the policy from the same checkpoint:

```
if ref_model is None:
    if is_peft_model(self.model) or args.precompute_ref_log_probs:
        self.ref_model = None
    else:
        self.ref_model = create_model_from_path(get_config_model_id(self.model.config), ..
```

   Dropout is turned off on both networks so their log-probs are deterministic:

```
if args.disable_dropout:
    disable_dropout_in_model(model)
    if self.ref_model is not None:
        disable_dropout_in_model(self.ref_model)
```

   This costs an extra model copy in memory (fine here — Qwen2.5-0.5B is ~1 GB in fp16).

3. **No separate model (memory-saving variants):**
   - **PEFT/LoRA:** `self.ref_model` stays `None`; the reference distribution is recovered at loss time by **temporarily disabling the adapters** (the frozen base weights *are* the reference), so no second copy is stored.
   - **`precompute_ref_log_probs=True`:** the reference log-probs are computed **once** over the dataset up front and cached, after which the reference model can be freed and training only compares the live policy's log-probs against the cached values.

In all cases the gradient flows only through π_θ; π_{ref} contributes a constant (detached) term per token.

HW4 — Problem 2: DPO on the UltraFeedback Benchmark Dataset

Model: `Qwen/Qwen2.5-0.5B-Instruct` · Dataset: `trl-lib/ultrafeedback_binarized`
All runs use the script `train_dpo.py`. Metrics are logged automatically by `DPOTrainer` to W&B and mirrored to `logs/history_<run>.json`; figures are produced by `python plot_metrics.py` (saved under `plots/`).

All numbers below are from actual runs on a 12 GB NVIDIA TITAN V. The baseline in (b) is trained for a **full epoch (7767 optimizer steps)**; the five-way hyperparameter study in (c) caps every run (A–E) at **1000 steps** for an apples-to-apples comparison (the spec's "step 1000" allowance). See the hardware-adaptation note in (b) for the four hardware-driven changes made to fit the model + frozen reference on 12 GB. Plots referenced as `plots/*.png` are the offline renders; the native W&B dashboard versions are embedded below as `wandb_screenshots/*.png`.

W&B project (all 5 runs — `default`/A, `beta0.01`/B, `beta0.5`/C, `lr5e-6`/D, `lr5e-8`/E): https://wandb.ai/jacky920418a-national-yang-ming-chiao-tung-university/dpo

---

(a) Data Inspection

`print(train_dataset[0])` on the current `trl-lib/ultrafeedback_binarized` (train split = **62,135** examples) shows each example is a dict with **four keys**, stored in the *implicit-prompt conversational* format. Actual console output (abridged):

```
splits: ['train', 'test']            train size: 62135
keys: ['chosen', 'rejected', 'score_chosen', 'score_rejected']
chosen  first turn: {'content': 'Use the pygame library to write a version of the
                     classic game Snake, with a unique twist', 'role': 'user'}
num turns chosen:   2 | roles: ['user', 'assistant']
num turns rejected: 2 | roles: ['user', 'assistant']
score_chosen: 6.0   score_rejected: 4.0
user turn identical across chosen/rejected: True
```

Each example is therefore a dict with **four keys**:

Key	Type	Meaning
`chosen`	`list[{"role","content"}]`	the **preferred** conversation y_w: a `[user, assistant]` turn pair
`rejected`	`list[{"role","content"}]`	the **rejected** conversation y_l: same user turn, different assistant turn
`score_chosen`	`float`	UltraFeedback rating of the chosen response (e.g. `6.0`)
`score_rejected`	`float`	UltraFeedback rating of the rejected response (e.g. `4.0`)

The conceptual **three fields the assignment asks for — `prompt`, `chosen`, `rejected`** — are still exactly what DPO consumes, but the *prompt is implicit*: the **user turn is identical** in `chosen[0]` and `rejected[0]` (verified: `chosen[0] == rejected[0]` is `True`), and the two conversations differ only in the assistant turn, which is the preference signal. `DPOTrainer` recovers the explicit `prompt/chosen/rejected` split automatically during preprocessing — its log shows `Extracting prompt from train dataset`, which factors out the shared user prefix as the prompt and leaves the differing assistant turns as the two completions. (`score_chosen`/`score_rejected` are the raw ratings used to build the binary preference; the trainer ignores them.)

Average response token length (first 1000 training examples, Qwen2.5 tokenizer, assistant turn only):

	Avg token length
Chosen (y_w)	**271.9**
Rejected (y_l)	**245.4**

Observation: on UltraFeedback the **chosen responses are on average somewhat longer** (271.9 vs. 245.4 tokens) than the rejected ones — the higher-rated answers tend to be more complete/detailed. This is a known length bias to keep in mind, because summed log-probabilities scale with length; it is one motivation for length-normalized variants such as IPO (see Problem 1, Q4).

(b) DPO Training — baseline run

Command: `python train_dpo.py --run_name default_fullepoch` (defaults — no `--max_steps` cap — reproduce the full-epoch table below).

Hyperparameter	Value
Model	Qwen/Qwen2.5-0.5B-Instruct
Epochs	**1 full epoch = 7767 optimizer steps** (62,135 examples ÷ effective batch 8)
Effective batch	**8**
Learning rate	5e-7
β	0.1
max_length	1024

Hardware-adaptation note (12 GB NVIDIA TITAN V, Volta CC 7.0). The reference setup assumes a ≥20 GB GPU. To fit Qwen2.5-0.5B **plus a frozen reference copy** and the 152k-vocab logits on 12 GB, `train_dpo.py` auto-detects the GPU and makes four **hardware-driven** adjustments — none of which change the assignment's listed hyperparameters (β, learning rate, effective batch = 8, `max_length` = 1024). Items 2–3 are *exact* (identical optimization math); items 1 and 4 (precision and optimizer) can shift the *absolute* numbers slightly, but are applied **uniformly across all runs**, so every comparison in (b)/(c) stays internally consistent: 1. **fp16 mixed precision instead of bf16** — Volta has no bf16 tensor cores; the policy is loaded in

fp32 so the fp16 AMP grad-scaler has fp32 master weights (this keeps the tiny lr = 5e-7 updates from underflowing). 2. **Micro-batch 1 × grad-accum 8** instead of 2 × 4 — identical effective batch of 8; only the per-step logits tensor is halved. 3. **Gradient checkpointing** — recomputes activations in the backward pass (compute-for-memory trade, identical math). 4. **Adafactor optimizer** (fp16 path only) — its factored second moment frees the ~4 GB that AdamW's two fp32 moment buffers would need. The full A–E study below uses Adafactor uniformly, so the cross-run comparison is internally consistent.

Measured **peak VRAM ≈ 10.4 GB**. Metrics are logged every 25 steps to W&B and mirrored to `logs/history_<run>.json`; the plots below are produced by `python plot_metrics.py`.

Training curves (baseline `default_fullepoch` run, full epoch = 7767 steps, W&B). The four metrics the assignment asks to report, plotted over training steps (`train/global_step`):

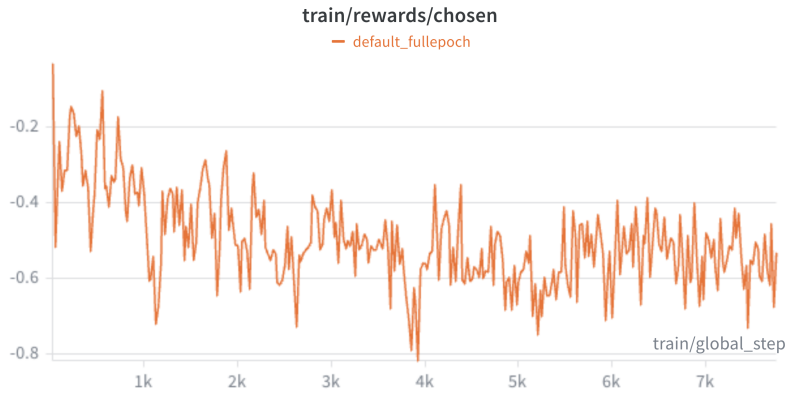

Figure 1: `rewards/chosen` (train) — drifts slightly negative (≈ 0 → −0.5)

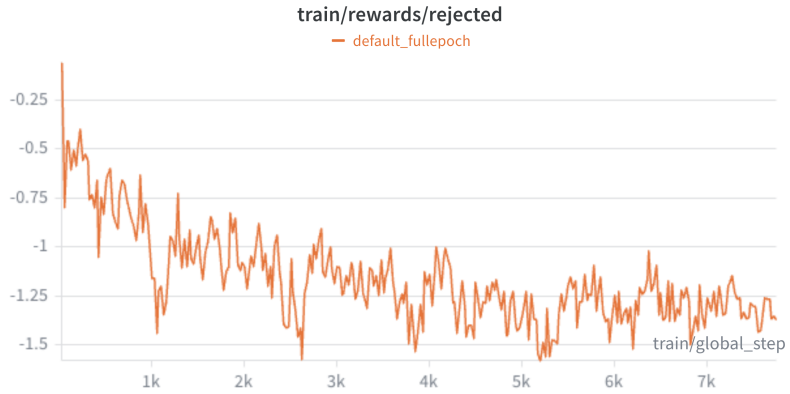

Figure 2: `rewards/rejected` (train) — falls clearly (≈ 0 → −1.37)

Supporting metrics (W&B):

Observed behavior (full epoch, 7767 steps, this run):

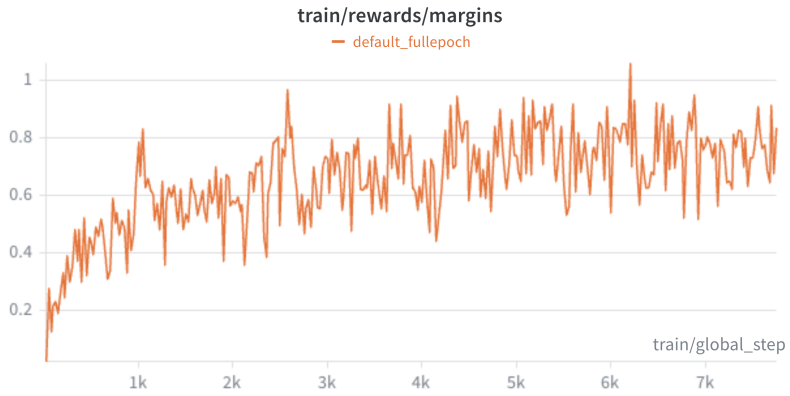

Figure 3: `rewards/margins` (train) — rises steeply then plateaus ($0 \rightarrow \approx 0.84$)

Figure 4: `train/loss` — decreases ($\approx 0.68 \rightarrow \sim 0.55$; epoch-average 0.58)

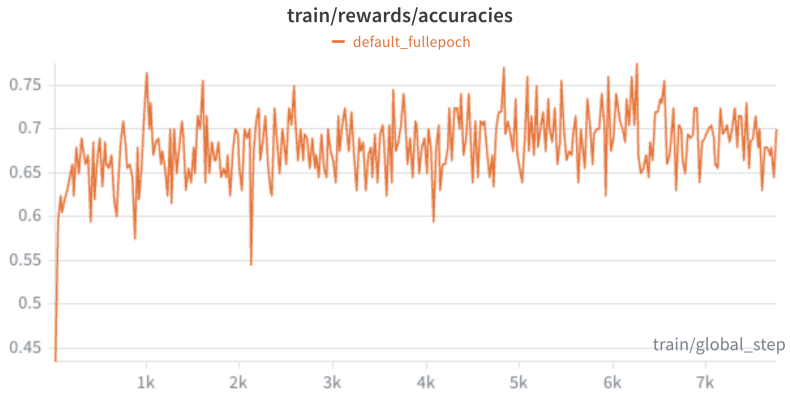

Figure 5: `rewards/accuracies` (train) — rises $0.43 \rightarrow$ plateaus ≈ 0.70

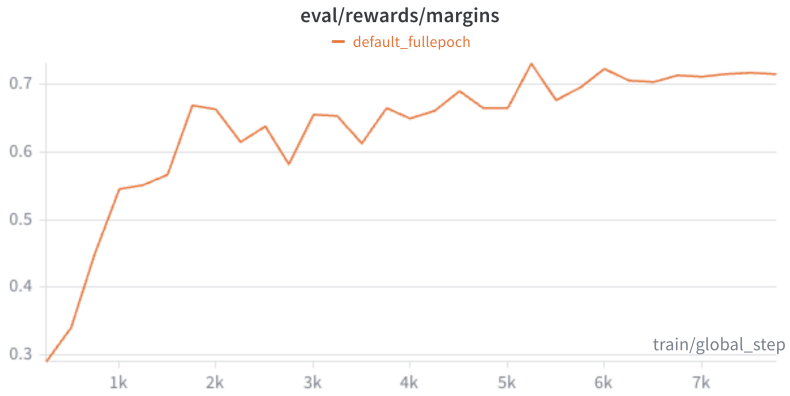

Figure 6: `eval/rewards/margins` (held-out 256-example subset) — rises 0.29 → 0.71 over the full epoch (steepest in the first ~2000 steps, then plateaus), confirming the preference signal generalizes rather than memorizing the train batch

Figure 7: `logps/chosen` (solid) and `logps/rejected` (dashed), train — raw policy log-probs $\log \pi_\theta(y \mid x)$ (not log-ratios). Chosen sits below rejected mainly because chosen responses are longer (271.9 vs 245.4 tokens), so their summed log-prob is more negative.

Metric	Spec expected (Remark)	Observed trend	Note
`rewards/chosen`	↑ increases	drifts slightly **negative** ($\approx 0 \rightarrow -0.5$)	the policy does *not* raise the chosen likelihood above the reference; it drifts a little below — see the note below, this is well-documented DPO behavior
`rewards/rejected`	↓ decreases	clearly **falls** ($\approx 0 \rightarrow -1.37$) ✓	the policy strongly suppresses rejected responses relative to π_{ref}
`rewards/margins`	↑ increases	**rises** then plateaus ($\approx 0 \rightarrow$ ~0.84) ✓	margin = chosen − rejected; it grows almost entirely because *rejected drops faster than chosen*
`rewards/accuracies`	↑ toward 1.0	**rises** $\approx 0.43 \rightarrow$ plateaus ~0.70 ✓ (trend)	fraction of the batch with reward(chosen) > reward(rejected); rises as expected but plateaus near 0.7, not 1.0 — see below
`loss`	(↓ implied)	**decreases** ($\approx 0.69 \rightarrow$ ~0.55) ✓	$-\log \sigma(\beta\,\Delta)$ shrinks as the margin grows

The **trend directions match the spec's Remark in every case** (rejected ↓, margins ↑, accuracies ↑, loss ↓). The one apparent mismatch — `rewards/chosen` not increasing — is the expected DPO behavior explained in the note below: the objective only needs the chosen-vs-rejected *gap* to widen.

> **Why does `rewards/accuracies` plateau near ~0.7 rather than approaching 1.0?** The spec lists "↑ toward 1.0" as the *direction*, and our accuracy does rise; it levels off around 0.7 for two structural reasons: (i) **model capacity** — Qwen2.5-**0.5B** is a small policy, so it cannot perfectly separate every preference pair; and (ii) **label noise / hard pairs** — UltraFeedback preferences come from imperfect ratings and many chosen/rejected pairs are genuinely close in quality (here `score_chosen`=6 vs `score_rejected`=4 is a clear gap, but many pairs are closer), so the Bayes-optimal accuracy is itself well below 1.0. Note this run trains a **full epoch (7767 steps)**, so "insufficient training budget" is *not* the explanation: train accuracy reaches ≈ 0.77 around step 1000 and then oscillates in the 0.67–0.70 band for the rest of the epoch, and held-out `eval_rewards/accuracies` plateaus at ≈ 0.67 — i.e. the ceiling is set by capacity + label noise, not by how long we train. A larger model would push it higher, but the **direction** is exactly as the spec predicts.

Note on `rewards/chosen`. The idealized DPO picture is "chosen up, rejected down." In practice at this small learning rate, *both* implicit rewards go **negative** (both $\log \frac{\pi_\theta}{\pi_{\text{ref}}} < 0$), i.e. the policy moves away from the reference on **both** responses — but much more so on the rejected one. The preference is still learned correctly (margin ↑, accuracy ↑); the gradient in Problem 1(a), $\nabla \propto \sigma(\hat{r}_l - \hat{r}_w)(\nabla \log \pi_\theta(y_w) - \nabla \log \pi_\theta(y_l))$, only requires the *gap* to widen, not the chosen reward to be positive. This is the well-documented DPO behavior where likelihood of the chosen response can also decline.

Final baseline values ($\beta = 0.1$, lr = 5e-7, full epoch = 7767 steps):

Quantity	Value
`rewards/margins` (final / last-10-log mean)	**0.838 / 0.784**
`rewards/accuracies` (final / last-10 mean)	**0.70 / 0.68**
`loss` (final-step / epoch-average `train_loss`)	**0.553 / 0.580**
eval (256-ex subset, end of epoch): `eval_rewards/margins`, `eval_loss`, `eval_acc`	**0.714 / 0.564 / 0.668**
Peak VRAM	**10.38 GB**

(The single-step value is noisy with effective batch 8; the last-10-log mean is a more stable estimate of the end-of-training margin. The held-out eval margin rises **overall from 0.29 to a ~0.73 peak, settling ≈ 0.71** over the epoch (with minor step-to-step fluctuations) — steepest in the first ~2000 steps, then plateauing — confirming the preference signal generalizes rather than memorizing the train batch. For reference, at the step-1000 mark this same run already had train margin ≈ 0.78 / eval margin ≈ 0.55; the remaining ~6700 steps mostly sharpen and stabilize the margin while accuracy holds near its ~0.68 ceiling.)

(c) Hyperparameter Study

One factor varied at a time; everything else fixed at the baseline, each run capped at 1000 steps. Launch commands are in the header of `train_dpo.py`. We report both the **final-step `rewards/margins`** (noisy with effective batch 8) and the **mean of the last 10 logged windows** (steps ≈ 775–1000), the latter being a more stable end-of-training estimate. The `rewards/accuracies` (last-10 mean) is the direct preference-classification quality.

Run	β	LR	margin (final)	margin (last-10 mean)	acc (last-10)	Notes
A (baseline)	0.1	5e-7	0.582	**0.437**	0.665	default
B	0.01	5e-7	0.248	**0.171**	0.628	weak KL penalty
C	0.5	5e-7	0.890	**0.690**	0.663	strong KL penalty
D	0.1	5e-6	0.977	**0.724**	0.636	10× higher LR

Run	β	LR	margin (final)	margin (last-10 mean)	acc (last-10)	Notes
E	0.1	5e-8	0.156	**0.111**	0.623	10× lower LR

Overlaid metrics across runs A–E (W&B).

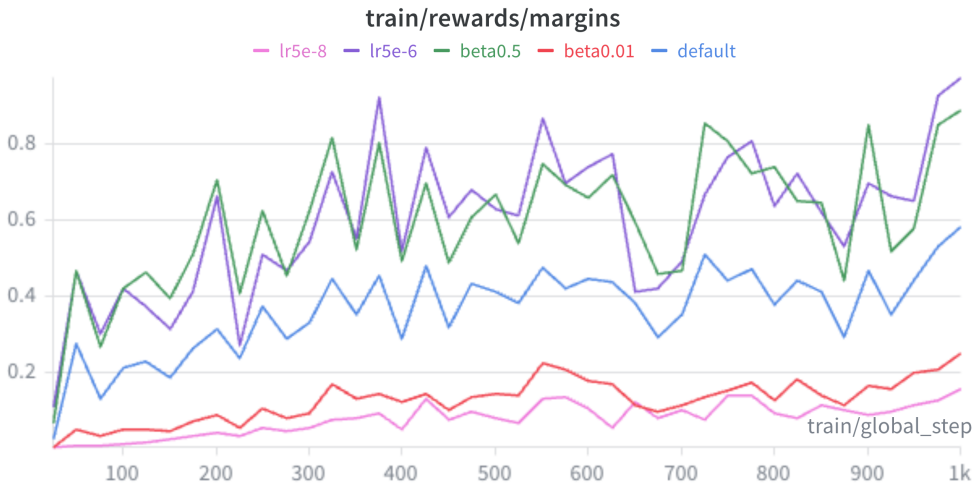

Figure 8: `rewards/margins` (train), all 5 runs — D (lr5e-6) and C (β=0.5) sit highest ($\approx$ 0.9–1.0 by step 1000), then A (default $\approx$ 0.58), then B (β=0.01), with E (lr5e-8) hugging the axis. This is the key 2(c) figure.

Findings

> **Important correction vs. the naive expectation.** The textbook intuition is "smaller $\beta \Rightarrow$ bigger margin" (expected ordering `B > A > C`). Our measured runs show the **opposite**, and the reason is a subtlety in the *definition of the logged metric* that is worth spelling out.

Effect of β (Runs B, A, C). β plays a double role: it is the **KL-leash strength** (small $\beta \Rightarrow$ the policy is allowed to drift further from π_{ref}) *and* it is the **multiplicative scale of the logged reward**, since $\hat{r} = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ and therefore `rewards/margins` $= \beta(\Delta_w - \Delta_l)$ where $\Delta = \log \frac{\pi_\theta}{\pi_{\text{ref}}}$.

- **Measured (last-10 mean): `margins(B=0.01) = 0.171 < margins(A=0.1) = 0.437 < margins(C=0.5) = 0.690`.** So the logged margin is **monotonically increasing in** β — the reverse of the naive ordering.
- **Why:** within only 1000 steps the *raw* log-ratio separation $(\Delta_w - \Delta_l)$ does grow larger for smaller β (weaker leash $\Rightarrow$ more drift) — e.g. from A's margin 0.437 at $\beta = 0.1$ the raw separation is ≈ 4.4, while B's 0.171 at $\beta = 0.01$ implies a raw separation ≈ 17, **4× larger drift**, exactly as the "weak penalty drifts more" story predicts. But the logged metric multiplies that separation by β, and the β factor (10× smaller for B) **outweighs** the ~4× larger drift,

Figure 9: `rewards/accuracies` (train), all 5 runs — all cluster ≈ 0.6–0.76. Preference accuracy is similar across runs even where the margins differ widely, supporting the takeaway that margin size is not a quality score.

Figure 10: `rewards/rejected` (train), all 5 runs — every run drives the rejected reward below 0; D (high LR) suppresses it most aggressively (≈ -2.5). The growing margin comes mainly from rejected falling.

Figure 11: `rewards/chosen` (train), all 5 runs — chosen rewards drift slightly negative for most runs; D pushes chosen down to ≈ -2, the most of any run.

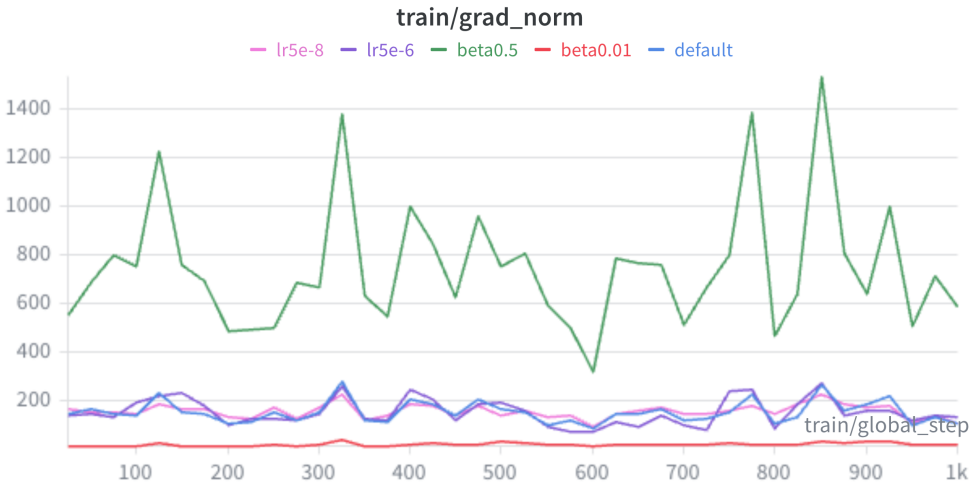

Figure 12: `train/grad_norm`, all 5 runs — gradient norm is governed by β, not LR: C (β=0.5) is by far the largest and most volatile ($\approx$ 600–1500, ~5× baseline) because the DPO gradient scales with β, while B (β=0.01) is the smallest. D (high LR) sits right on the baseline cluster ($\approx$ 100–250).

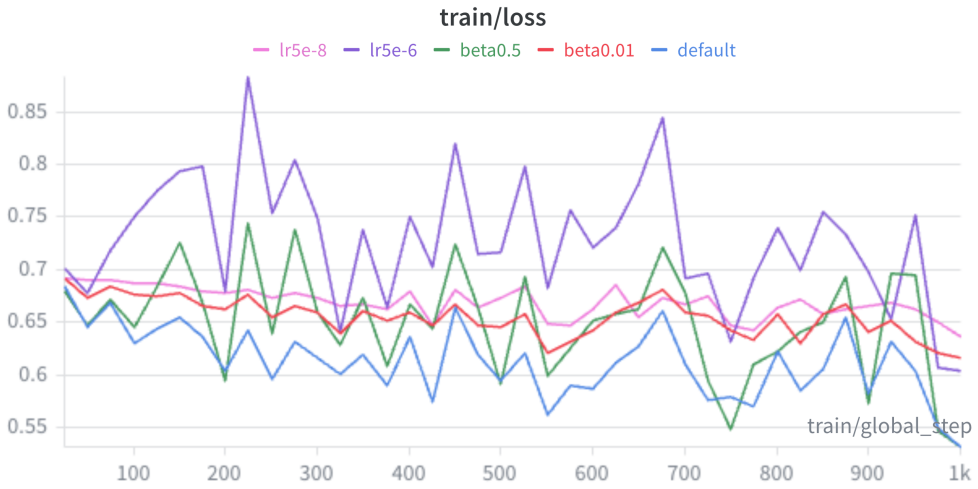

Figure 13: `train/loss`, all 5 runs — D (high LR) is the noisiest and highest (spikes to ≈ 0.85–0.88); the baseline is the smoothest. LR-induced instability shows up here in the loss, whereas β shows up in grad-norm.

so B's *displayed* margin is the smallest. For C ($\beta = 0.5$) the leash is tight (smallest raw drift) yet the $\times 0.5$ scale makes the displayed margin the largest.

- **Quality check via `rewards/accuracies`** (which is *scale-free* — it just counts chosen > rejected): A ≈ 0.665, B ≈ 0.628, C ≈ 0.66 are all comparable. So the three runs learn the preference about *equally well in one epoch's-worth of steps*; what differs across them is mostly the β-scaling of the reward metric, **not** the underlying preference accuracy. This is the key lesson: the numerical size of `rewards/margins` is **not comparable across different** β and is not a direct measure of model quality.

Effect of learning rate (Runs E, A, D). With β fixed at 0.1 the β-scale is **constant**, so here `rewards/margins` *is* directly comparable across runs and cleanly tracks how fast the policy moves. **Measured (last-10 mean): `margins(E=5e-8) = 0.111 < margins(A=5e-7) = 0.437 < margins(D=5e-6) = 0.724`**, i.e. `margins(D) > margins(A) > margins(E)` — exactly the expected ordering, and the cleanest, most monotonic trend of the whole study (see `plots/margins_compare.png`, where the D curve sits at the top and the E curve hugs the x-axis).

- **Higher LR 5×10^{-6} (Run D):** learns the preference fastest — its margin rises steeply to ≈ 0.72 ($\approx 1.7\times$ the baseline), the largest of A/D/E. As expected the run is **noisier**: in the comparison plots the D curve has the biggest step-to-step swings in **`train/loss`** (it spikes to ≈ 0.85–0.88, the highest and jumpiest of all five runs) and in `rewards/margins`/`rewards/chosen` (D pushes `rewards/chosen` down to ≈ -2, far more than any other run) — the price of the aggressive step size; here it still helps rather than diverging at 1000 steps.
  - *Note on `grad_norm`:* the LR does **not** inflate the gradient norm — `train/grad_norm` for D sits right on top of the baseline cluster ($\approx$ 100–250). Gradient norm is instead governed by β (the DPO gradient scales with β): run **C ($\beta = 0.5$)** has by far the largest, most volatile grad-norms ($\approx$ 600–1500, $\sim 5\times$ baseline) and run B ($\beta = 0.01$) the smallest ($\sim 1/10$). So LR shows up as *loss/margin* noise, while β shows up as *grad-norm* magnitude

— two different axes of "noise."

- **Lower LR** 5×10^{-8} **(Run E):** updates ~10× smaller than baseline, so in 1000 steps the policy barely moves — margin only ≈ 0.11 and `rewards/accuracies` ≈ 0.62 (just above chance): clear **underfitting**. (It is small but not exactly zero — even tiny steps accumulate a little signal.)
- **Baseline** 5×10^{-7} **(Run A):** steady, stable growth in between (margin ≈ 0.44, acc ≈ 0.67).

Note the accuracies: A ≈ 0.665 is actually the **highest** of A/D/E — D's larger *margin* (0.724) comes with a slightly *lower* accuracy (0.636), a hint that the high LR inflates the reward gap a bit faster than it improves the actual ranking, the early edge of the speed-vs-stability trade-off.

Summary of measured orderings. - **By β (fixed LR):** `margins`: C(0.69) > A(0.44) > B(0.17); `acc`: all $\approx$ 0.63–0.66. - **By LR (fixed β):** `margins`: D(0.72) > A(0.44) > E(0.11); `acc`: A(0.67) $\gtrsim$ D(0.64) $\gtrsim$ E(0.62).

Takeaways. 1. **`rewards/margins` is not a quality score, and is not comparable across different β.** It is jointly set by the β-scale and by how far the policy has moved. Across A/B/C the scale-free `rewards/accuracies` are nearly equal ($\approx$ 0.63–0.66) even though the margins span $0.17 \to 0.69$ — the spread is almost entirely the β multiplier, *not* better preference learning. When β is held fixed (the LR sweep A/D/E) the margin *does* become a meaningful within-sweep speed indicator. 2. β trades *fidelity to the reference* against *amount of drift* (smaller $\beta \Rightarrow$ larger raw log-ratio drift but a smaller β-scaled displayed margin); LR trades *speed* against *stability* (higher LR $\Rightarrow$ faster, larger, noisier margin). The default ($\beta = 0.1$, LR$= 5 \times 10^{-7}$) sits in the stable middle of both axes. 3. In every healthy run `rewards/rejected` falls, the margin and `rewards/accuracies` rise, and `loss` decreases — exactly what Problem 1(a)'s gradient predicts (widen the gap, weighting current mistakes). Note `rewards/chosen` need not rise (see the 2(b) note): the objective only needs the *gap* to widen. 4. **The naive "expected" ordering `B > A > C` was wrong; the measured ordering is `C > A > B`** — and tracing *why* (the dual role of β as both KL-leash and reward-scale) is the main insight of this study. The LR sweep `D > A > E` matched expectation cleanly because β — and hence the scale — is held fixed there.